

Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Радіотехнічний факультет
Кафедра прикладної радіоелектроніки

Звіт
про лабораторну роботу №2 за темою
«Шифрування та розшифрування на машині «Енігма»»
з дисципліни
«Захист даних»

Викладач:

Навроцький Денис Олександрович

Виконав:

Студент групи РС-61мн

Гандзюк Дмитро Васильович

Київ – 2026 р.

Лабораторна робота №2

Шифрування та розшифрування на машині «Енігма»

Мета роботи: реалізувати шифр, побудований на двох базових криптографічних примітивах — перестановці (зсуві) та заміні (підстановці), — на прикладі спрощеної моделі шифрувальної машини «Енігма», довести коректність реалізації на офіційних валідаторах навчальної платформи CodinGame, а також самостійно оцінити криптографічну стійкість отриманого шифру: дослідити статистичні властивості шифротексту, визначити фактичний розмір простору ключів і практично реалізувати атаки в усіх основних моделях зломисника — від перебору ключа до відновлення відкритого тексту лише за шифротекстом.

Хід роботи

Постановка задачі

Задача взята з навчальної платформи CodinGame (розділ Training, рівень складності Easy) і має назву «Encryption/Decryption of Enigma Machine». У ній відтворено спрощений принцип роботи шифрувальної машини «Енігма», у якому обидва класичні криптографічні примітиви подані в чистому, нічим не замаскованому вигляді.

Перший примітив — **перестановка алфавіту**, реалізована як шифр Цезаря з інкрементним ключем: перший символ повідомлення зсувається на N позицій, другий — на $N+1$, третій — на $N+2$ і так далі. Другий примітив — **заміна**: отриманий після зсуву текст послідовно пропускається крізь три ротори, кожен з яких є довільною підстановкою (бієкцією) 26-літерного латинського алфавіту.

Формат вхідних даних задачі:

рядок 1	ENCODE DECODE
рядок 2	початковий зсув N , $0 \leq N < 26$
рядки 3–5	три ротори по 26 великих латинських літер
рядок 6	повідомлення, $1 \leq \text{довжина} < 50$, лише A..Z

Результатом роботи програми має бути одне слово — зашифроване або розшифроване повідомлення. Наведений в умові приклад зручно розібрати покроково: для повідомлення AAA та початкового зсуву $N = 4$ зсув дає EFG

(A+4, A+5, A+6), перший ротор перетворює це на JLC, другий — на BHD, третій — на KQF. Саме KQF і є очікуваною відповіддю.

Математична модель шифру

Позначимо літери числами від 0 до 25 ($A = 0$, $Z = 25$), а всі операції виконуватимемо в кільці лишків за модулем 26. Нехай $R1$, $R2$, $R3$ — підстановки, задані роторами, N — початковий зсув, m — відкритий текст, c — шифротекст. Тоді пряме та зворотне перетворення записуються так:

$$\begin{aligned}c[i] &= S[(m[i] + N + i) \bmod 26], & S &= R3 \cdot R2 \cdot R1 \\m[i] &= (S^{-1}[c[i]] - N - i) \bmod 26\end{aligned}$$

З цього запису впливають три спостереження, на яких надалі тримається вся дослідницька частина роботи.

Спостереження 1. Три ротори — це один ротор. Композиція бієкцій є знову бієкцією, тому добуток $S = R3 \cdot R2 \cdot R1$ — це одна підстановка алфавіту. Каскад із трьох роторів не дає нічого такого, чого не дав би один належно підібраний ротор. Це твердження перевірено не міркуванням, а програмно: метод `equivalent_single_rotor()` будує машину з одним ротором і нульовим зсувом, а модульний тест порівнює шифротексти обох машин на 50 випадкових конфігураціях — вони збігаються символ у символ.

Спостереження 2. Шифр є поліалфавітним із періодом рівно 26. На позиції i застосовується алфавіт заміни з номером $p = (N + i) \bmod 26$, причому всі 26 алфавітів породжені однією підстановкою S за правилом $A_p(m) = S[(m + p) \bmod 26]$. Отже, послідовність алфавітів циклічно повторюється з періодом 26 незалежно від вмісту повідомлення.

Спостереження 3. Увесь ключ стискається до 26 чисел. Введемо поняття *ефективного ключа* D , визначивши $D[x] = (S^{-1}[x] - N) \bmod 26$. Тоді розшифрування набуває вигляду $m[i] = (D[c[i]] - i) \bmod 26$. Це повний опис шифру: початковий зсув N «втоплено» всередину підстановки і окремо його знати не потрібно. Саме ефективний ключ D відновлюють усі чотири реалізовані далі атаки.

Програмна реалізація

Розв'язок виконано мовою Python 3 (основна реалізація) і продубльовано мовою C# на платформі .NET 8 (незалежна реалізація для

перехресної перевірки). Вихідний код самодостатнього розв’язку для платформи наведено в додатку А, ядро шифру — у додатку Б, модуль криптоаналізу — у додатку В, реалізацію мовою С# — у додатку Г.

Головне архітектурне рішення випливає зі спостереження 1: каскад роторів згортається в одну таблицю підстановки до початку обробки повідомлення. Завдяки цьому у внутрішньому циклі на кожен символ припадає лише одне звертання до таблиці. У повній версії бібліотеки попередньо обчислюються всі 26 алфавітів заміни (матриця 26×26 , тобто 676 байт), тому внутрішній цикл узагалі не містить арифметичних операцій — тільки індексацію.

Обчислювальна складність шифрування становить $O(L + 26k)$, де L — довжина повідомлення, а k — кількість роторів; додаткова пам’ять — $O(26^2)$ і не залежить від довжини повідомлення. Розшифрування симетричне та має ту саму складність. Ядро шифру й усі атаки написані виключно на стандартній бібліотеці Python, без зовнішніх залежностей.

Верифікація розв’язку

Правильність реалізації підтверджено на чотирьох незалежних рівнях: офіційними валідаторами платформи, збереженими в репозиторії офіційними тестами, набором властивісних тестів і перехресним порівнянням двох реалізацій різними мовами.

Розв’язок надіслано на перевірку платформі CodinGame. Усі шість видимих тестів пройдено, і, що важливіше, пройдено приховані валідатори: підсумкова оцінка становить **100 %**. Надісланий код у вікні редактора платформи наведено на рисунку 2.1, результат прогону всіх шести тестів — на рисунку 2.2, підсумкову оцінку валідаторів — на рисунку 2.3.

```

1
2 import sys
3
4 A = ord("A")
5 M = 26
6
7
8 def main() -> None:
9     data = sys.stdin.read().split()
10    mode = data[0].upper()
11    shift = int(data[1]) % M
12    rotors = data[2:5]
13    message = data[5] if len(data) > 5 else ""
14
15    # Collapse the rotor cascade into one substitution table.
16    s = list(range(M))
17    for wiring in rotors:
18        table = [ord(ch) - A for ch in wiring]
19        s = [table[x] for x in s]
20
21    if mode == "ENCODE":
22        out = [
23            chr(A + s[(ord(ch) - A + shift + i) % M])
24            for i, ch in enumerate(message)
25        ]
26    else:
27        inv = [0] * M
28        for i, v in enumerate(s):
29            inv[v] = i
30        out = [
31            chr(A + (inv[ord(ch) - A] - shift - i) % M)
32            for i, ch in enumerate(message)
33        ]
34
35    print("".join(out))
36
37
38 if __name__ == "__main__":
39     main()
40

```

Рисунок 2.1 – Надісланий розв’язок у вікні редактора CodinGame

Test cases			
01	Encode 3		▶ PLAY TESTCASE
02	Encode 23		▶ PLAY TESTCASE
03	Decode 21		▶ PLAY TESTCASE
04	Encode 21		▶ PLAY TESTCASE
05	Encode 42		▶ PLAY TESTCASE
06	Decode 49		▶ PLAY TESTCASE

Рисунок 2.2 – Усі шість тестів платформи позначені як пройдені

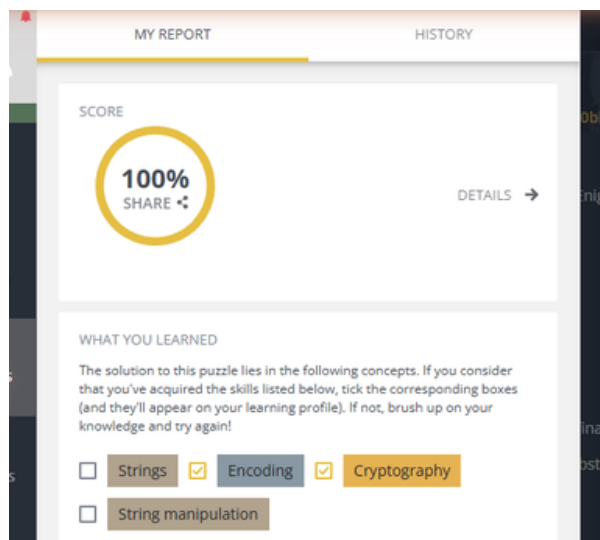


Рисунок 2.3 – Звіт платформи: оцінка валідаторів — 100 %

Входи та еталонні виходи всіх шести офіційних тестів збережено в репозиторії у файлі `tests/official_cases.json`, тому їх можна прогнати локально без доступу до платформи. Перелік тестів наведено в таблиці 2.1.

Таблиця 2.1 – Офіційні тести CodinGame

№	Назва	Режим	N	Довжина	Результат
1	Encode 3	ENCODE	4	3	пройдено
2	Encode 23	ENCODE	7	23	пройдено
3	Decode 21	DECODE	9	21	пройдено
4	Encode 21	ENCODE	9	21	пройдено
5	Encode 42	ENCODE	9	42	пройдено
6	Decode 49	DECODE	5	49	пройдено

Результат локального прогону цих тестів:

```
$ python run.py selftest
Офіційні тести CodinGame
```

```
-----
[OK] Encode 3      ENCODE -> KQF
[OK] Encode 23     ENCODE -> ALWAURKQEQQWLRAWZHUYKVN
[OK] Decode 21     DECODE -> EVERYONEISWELCOMHERE
[OK] Encode 21     ENCODE -> PQSACVVTOISXFXCIAMQEM
[OK] Encode 42     ENCODE -> PQSACVVTOISXFXCIAMQEMDZIXFJJSTQIENEFQXVZYV
[OK] Decode 49     DECODE -> THEQUICKBROWNFOXJUMPSOVERALAZYSphinxOFBLACKQUARTZ
-----
```

Пройдено 6 з 6

Окрім офіційних тестів, написано набір із 86 автоматичних тестів, що перевіряють: оборотність перетворення на 300 випадкових конфігураціях; періодичність шифру (перші 26 символів шифротексту рядка з 52 літер А збігаються з наступними 26); еквівалентність каскаду з трьох роторів одному ротору; модульність зсуву ($N = 3$ і $N = 29$ дають однаковий результат); некомутативність порядку роторів; відхилення некоректних роторів і символів поза алфавітом А..Z. Реалізація мовою С# проходить ті самі шість офіційних тестів і 2000 перевірок оборотності, а окремий тест порівнює вихід обох реалізацій символ у символ. Увесь набір виконується автоматично при кожній зміні коду засобами GitHub Actions на трьох версіях Python.

Під час підготовки роботи в самому тестовому наборі було виявлено й усунуто помилку: вивід програми на С# читався в кодуванні консолі Windows замість UTF-8, через що порівняння з українським рядком не спрацьовувало. Це показове нагадування, що тест, який ніколи не запускався, не є доказом

нічого.

Частотний аналіз шифротексту

Перше, що перевіряє криптоаналітик, — чи зберігає шифротекст частотну структуру мови. Для експерименту взято англomовний текст обсягом 2035 літер і зашифровано його з початковим зсувом $N = 7$. Порівняння частот літер відкритого тексту, шифротексту й еталонних частот англійської мови наведено на рисунку 2.4.

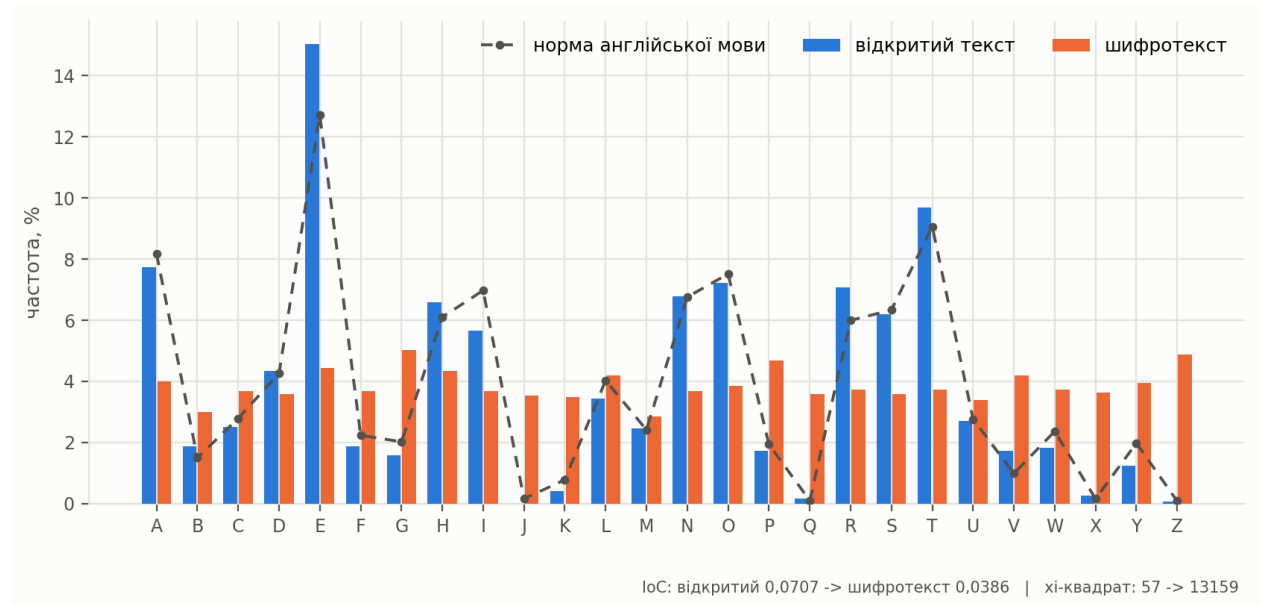


Рисунок 2.4 – Інкрементний зсув вирівнює частоти літер шифротексту

Результат на перший погляд свідчить на користь шифру. Інкрементний зсув «розмазує» кожен літеру відкритого тексту по всіх 26 позиціях алфавіту, і розподіл літер шифротексту стає майже рівномірним: максимальне відхилення частоти будь-якої літери від рівномірних 3,85 % становить лише 1,17 відсоткового пункту. Кількісні показники зведено в таблиці 2.2.

Таблиця 2.2 – Статистичні характеристики тексту до та після шифрування

Показник	Відкритий текст	Шифротекст	Орієнтир
Індекс відповідності (ІоС)	0,0707	0,0386	англ. 0,0667 / випадк. 0,0385
Критерій χ^2 -квадрат	57	13159	менше — ближче до мови
Ентропія, біт/символ	4,105	4,689	максимум 4,700

Індекс відповідності впав з 0,0707 (типове значення для англійської мови) до 0,0386, що практично збігається зі значенням для рівномірно випадкової послідовності. Ентропія зросла до 4,689 біта на символ при теоретичному максимумі 4,700. Класичний частотний аналіз тут безсилий: шифротекст статистично невідрізнити від шуму.

Саме на цьому місці й виникає найпоширеніша помилка — висновок, що шифр стійкий. Наступні розділи показують, що це не так.

Визначення періоду шифру

Ключ до злому дає індекс відповідності, застосований не до всього тексту, а до його частин. Розіб'ємо шифротекст на p стовпців за індексом $i \bmod p$. Якщо припущений період p збігається зі справжнім, то кожен стовпець виявиться результатом однієї й тієї самої моноалфавітної заміни, а індекс відповідності всередині стовпця повернеться до значення, характерного для природної мови (ІоС інваріантний щодо простої заміни). Профіль залежності середнього ІоС стовпця від припущеного періоду наведено на рисунку 2.5.

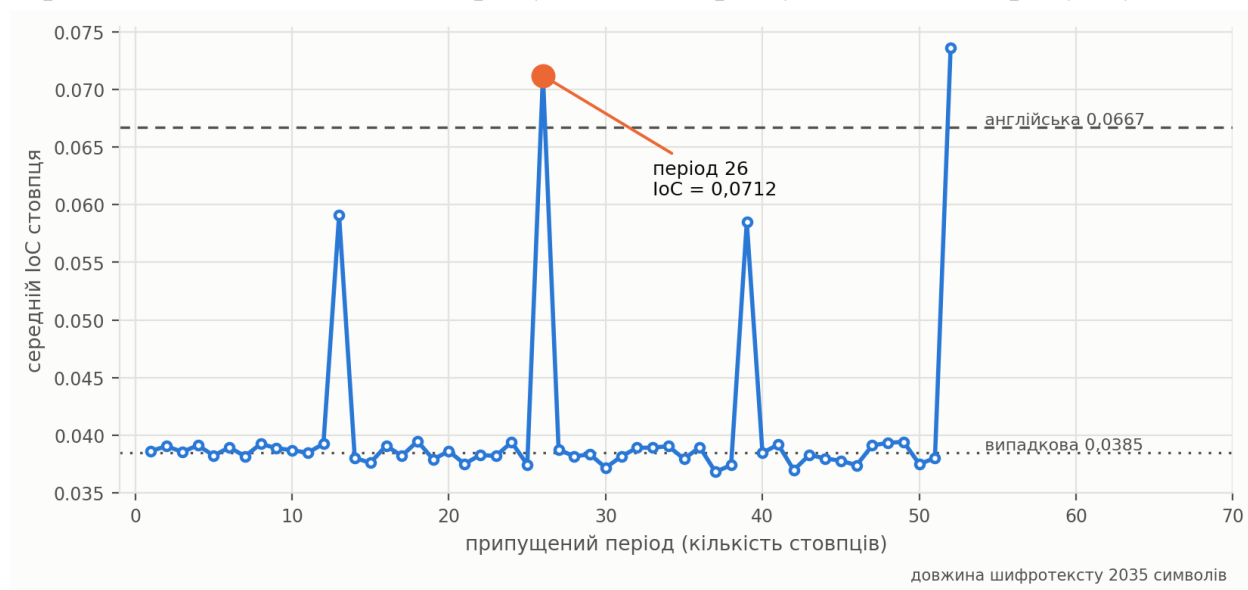


Рисунок 2.5 – Профіль індексу відповідності викриває поліалфавітний період 26

На профілі чітко видно сплеск рівно на періоді 26 (ІоС = 0,0712 проти 0,0386 для тексту в цілому) та на кратних йому значеннях 13, 39 і 52. Це експериментальне підтвердження спостереження 2, отримане без будь-якого знання про ротори чи початковий зсув. Отже, перший крок криптоаналізу — визначення структури шифру — виконується автоматично.

Ієрархія атак на шифр

Реалізовано чотири атаки, впорядковані за спаданням припущень про можливості зловмисника — від найсильніших до найслабших. Усі вони відновлюють один і той самий об'єкт — ефективний ключ D. Для оцінювання правдоподібності кандидатів використано статистичну модель англійської мови на квадриграмах, натреновану на корпусі обсягом 5 623 458 літер текстів суспільного надбання (у репозиторії зберігається лише похідна статистика частот, а не самі тексти). Зведені результати наведено в таблиці 2.3.

Таблиця 2.3 – Результати атак на повідомленні завдовжки 180 символів

Атака	Що відомо зловмиснику	Вартість	Час	Ключ
Перебір зсуву	ротори	26 варіантів	0,007 с	відновлено
Відомий відкритий текст	пара текстів	180 символів	< 0,001 с	відновлено
Підібраний відкритий текст	доступ до оракула	1 запит (26 символів)	< 0,001 с	відновлено
Тільки шифротекст	нічого, крім шифротексту	78 481 оцінка	3,04 с	відновлено

Атака 1 — повний перебір зсуву

Найважливіше зауваження до цієї задачі випливає з принципу Керкгоффа: стійкість шифру має залежати виключно від ключа, а не від секретності алгоритму. У розглянутій задачі ротори надруковані просто в умові, і, що показово, в усіх шести офіційних тестах вони однакові (це зафіксовано окремим тестом). Отже, справжнім секретом лишається єдине число — початковий зсув N.

Це дає лише 26 варіантів ключа, тобто близько 4,7 біта ентропії. Атака полягає в розшифруванні шифротексту всіма 26 ключами та виборі найбільш «англійського» результату. Розподіл оцінок правдоподібності по кандидатах наведено на рисунку 2.6.

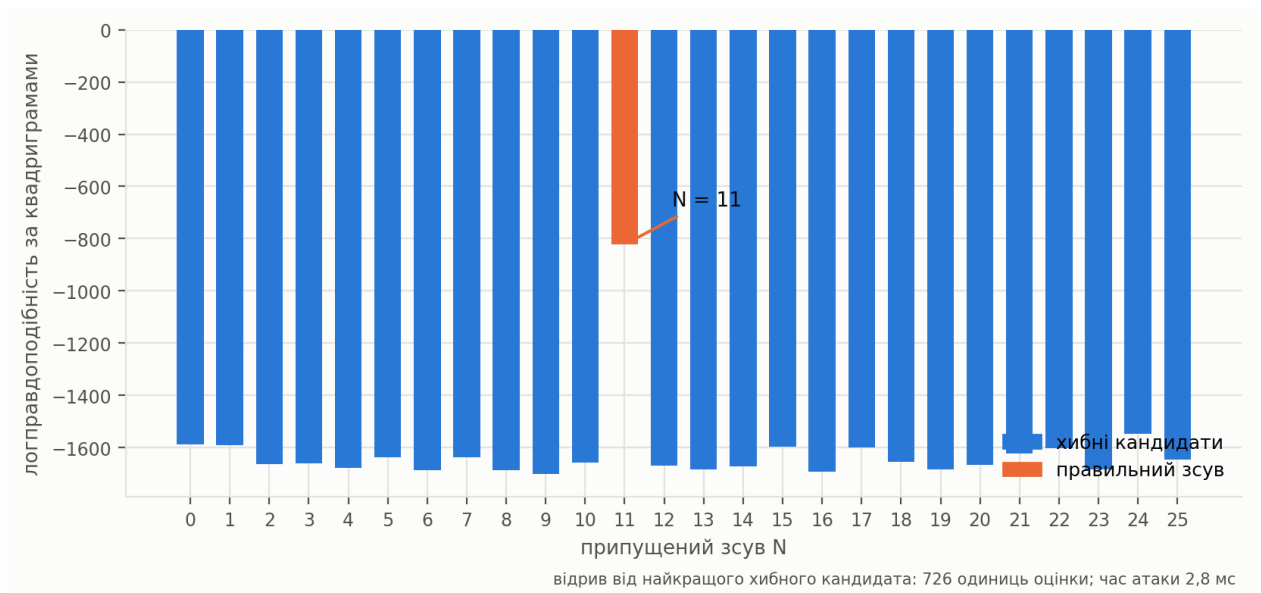


Рисунок 2.6 – За відомих роторів правильний ключ визначається однозначно з першої спроби

Перебір завершується за 2,8 мілісекунди, а правильний кандидат відривається від найкращого хибного на 726 одиниць логарифмічної правдоподібності, тож жодної неоднозначності не виникає. Атака успішна на 100 % повідомлень будь-якої довжини, включно з трисимвольними, — тобто в умовах цієї задачі шифр зламується повністю й миттєво.

Атака 2 — відомий відкритий текст

Якщо зловмиснику відомий хоча б фрагмент відкритого тексту разом з відповідним фрагментом шифротексту (так звана «шпаргалка», *crib*), ключ відновлюється взагалі без перебору: кожна відома пара символів дає один елемент ефективного ключа за формулою $D[c[i]] = (m[i] + i) \bmod 26$.

Виникає природне запитання: скільки відомого тексту для цього потрібно? Задача зводиться до класичної задачі про збирання купонів, для якої очікуване покриття описується формулою $26 \cdot (1 - (25/26)^L)$. Експериментальна перевірка (по 200 прогонів на кожну точку) показала збіг із теорією із середнім відхиленням 0,061 елемента ключа — рисунок 2.7 і таблиця 2.4.

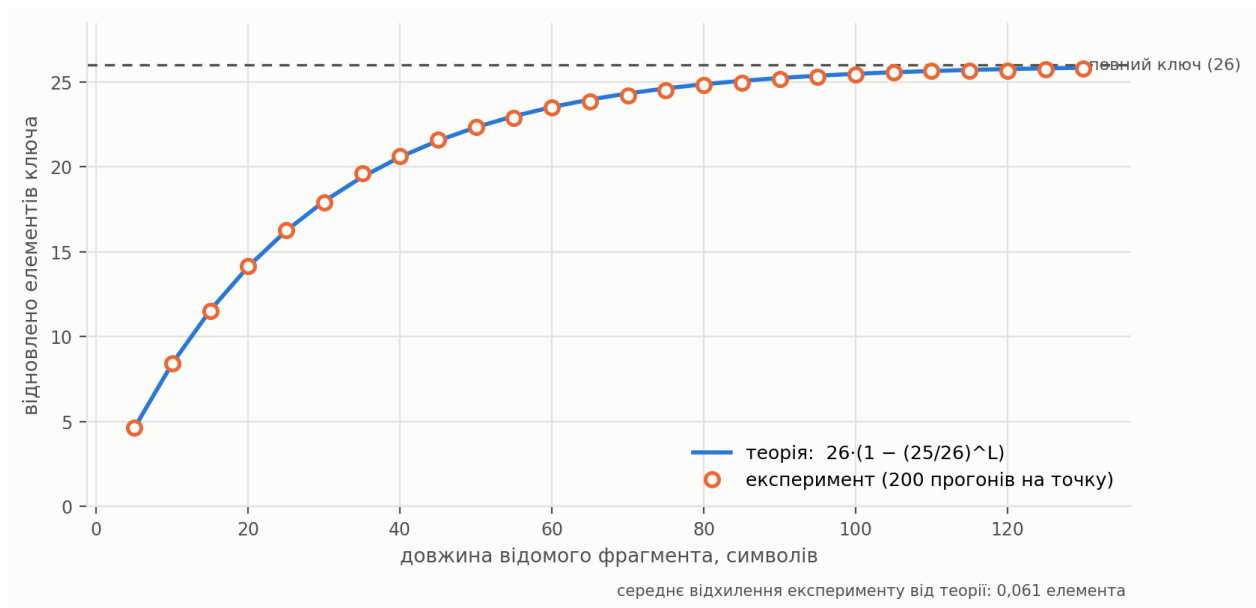


Рисунок 2.7 – Відновлення ключа за відомим відкритим текстом:
експеримент проти теорії

Таблиця 2.4 – Покриття ефективного ключа залежно від довжини відомого фрагмента

Довжина фрагмента L	10	25	50	105
Експеримент	8,44	16,29	22,35	25,56
Теорія	8,44	16,25	22,34	25,58

Щоб зібрати всі 26 елементів ключа, потрібно в середньому близько $26 \cdot H(26) \approx 100$ символів відомого тексту. Проте повний ключ зазвичай і не потрібен: навіть половина елементів дозволяє прочитати більшу частину повідомлення, а решту символів легко відновити за контекстом.

Атака 3 — підібраний відкритий текст

Найдешевша з реалізованих атак. Припустимо, що зловмисник має доступ до шифрувального пристрою як до «чорної скриньки» (оракула) і може подати на вхід довільне повідомлення. Надішлемо рядок із 26 однакових літер A. Тоді $c[i] = S[(0 + N + i) \bmod 26]$, а отже $D[c[i]] = i$ для всіх i від 0 до 25.

Оскільки S — бієкція, усі 26 отриманих символів різні, і повний ефективний ключ відновлюється з єдиного запиту завдовжки **26 символів**. Ані ротори, ані початковий зсув при цьому знати не потрібно. Коректність атаки перевірено на 25 випадкових конфігураціях машини — у кожному

випадку відновлений ключ точно збігся зі справжнім.

Атака 4 — відновлення тексту лише за шифротекстом

Це найслабша модель зломисника і, відповідно, найцікавіший результат роботи: відомий лише шифротекст, ротори й зсув невідомі повністю. Ключова ідея полягає в тому, що позиційний зсув *-і є публічною структурою, а не секретом*: його не треба вгадувати. Тому перебирати доводиться лише одну бієкцію D , і задача зводиться до звичайної моноалфавітної заміни.

Реалізовано сходження за схилом (hill climbing) у просторі бієкцій з фітнес-функцією на квадриграмах. Початкове наближення будується жадібним частотним методом: для кожної пари «літера шифротексту — кандидат» обчислюється частотна правдоподібність породжених символів, після чого пари розбираються за спаданням вигоди зі збереженням бієктивності. Далі ключ уточнюється випадковими обмінами з кількома незалежними перезапусками. Залежність частки повних зламів від довжини повідомлення наведено на рисунку 2.8 і в таблиці 2.5.

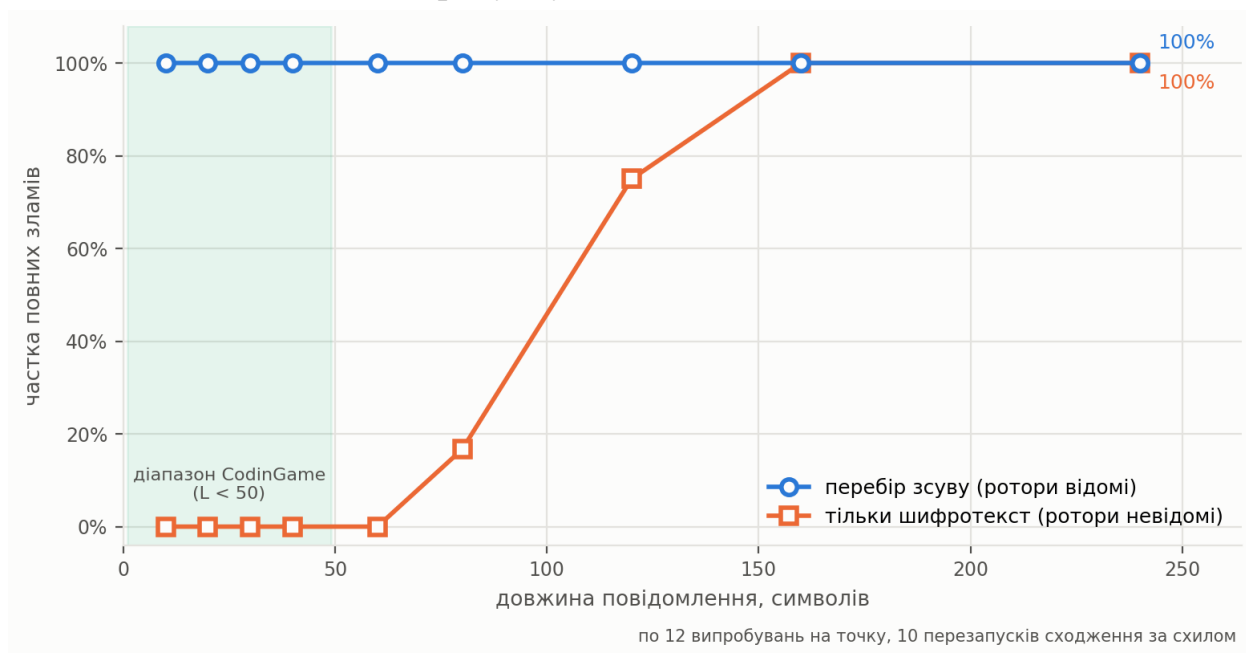


Рисунок 2.8 – Успішність атак залежно від довжини повідомлення

Таблиця 2.5 – Успішність атаки лише за шифротекстом

Довжина повідомлення, символів	40	80	120	160	240
Частка повних зламів, %	0	17	75	100	100

Довжина повідомлення, символів	40	80	120	160	240
Середній час атаки, с	0,24	0,49	0,74	1,20	1,06

Поріг має природне пояснення. Відстань єдиності за Шенноном для ключа обсягом $26!$ обчислюється як $U = H(K)/D = 88,4 / (4,70 - 1,5) \approx 28$ символів. Це означає, що шифротексти, коротші за приблизно 28 символів, у принципі мають кілька осмислених розшифрувань і не можуть бути однозначно зламані жодним алгоритмом. Отриманий практичний поріг (близько 120–160 символів) у 4–6 разів вищий за теоретичну межу, що є типовим розривом між інформаційно-теоретичною оцінкою та можливостями конкретного евристичного алгоритму.

Окремо варто зазначити наслідок для умови задачі: платформа обмежує довжину повідомлення 50 символами, тобто саме в тому діапазоні, де ціліснотекстова атака ще не працює надійно. Проте це не рятує шифр, бо ротори в задачі є публічними, і атака 1 ламає 100 % таких повідомлень за мілісекунди.

Оцінка простору ключів

Номінально ключем шифру є три довільні підстановки 26-літерного алфавіту та початковий зсув, тобто $(26!)^3 \cdot 26$ варіантів, що відповідає $2^{269,8}$. Проте, як показано у спостереженнях 1 і 3, весь цей набір згортається в одну бієкцію D , і фактичний простір ключів становить лише $26!$, тобто $2^{88,4}$. Якщо ж урахувати, що ротори опубліковані в умові, залишається 26 варіантів — $2^{4,7}$. Порівняння наведено на рисунку 2.9 і в таблиці 2.6.

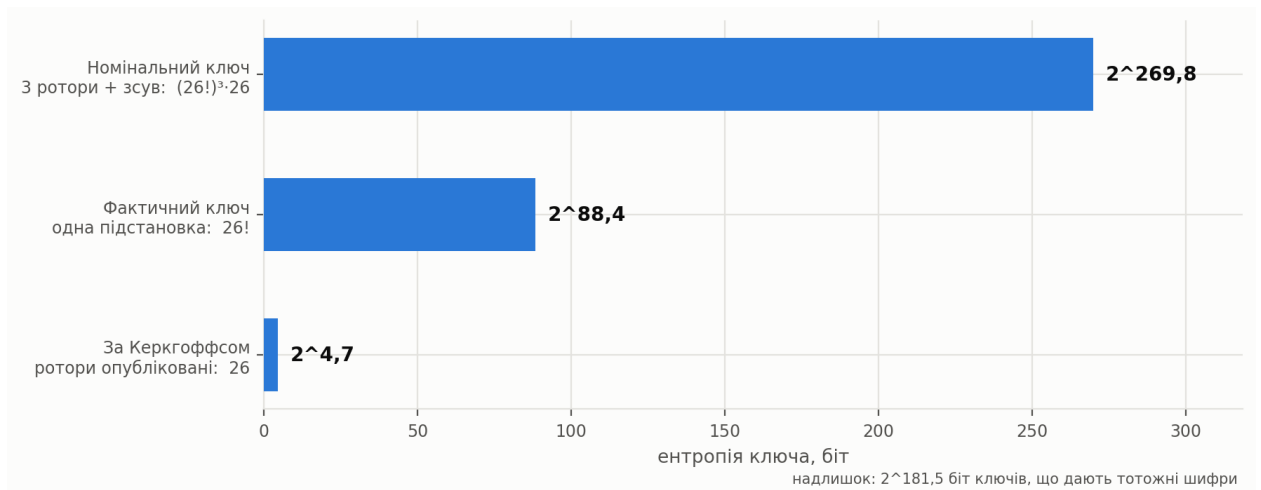


Рисунок 2.9 – Заявлений і фактичний розмір простору ключів

Таблиця 2.6 – Розмір простору ключів у різних моделях

Модель	Простір ключів	Ентропія ключа
Номинально: 3 ротори та зсув	$(26!)^3 \cdot 26$	$2^{269,8}$
Фактично: одна підстановка D	$26!$	$2^{88,4}$
За Керкгоффсом: ротори відомі	26	$2^{4,7}$

Отже, з заявлених $2^{269,8}$ біта корисними є лише $2^{88,4}$: рівно $2^{181,5}$ різних наборів «ротори + зсув» дають тотожні шифротексти. Це твердження доведено не лише формулою, а й конструктивно: функція `find_key_collision()` будує два різні ключі, шифротексти яких збігаються символ у символ на 120 випадкових літерах.

Дослідження дифузії

Для стійкого шифру виконується строгий лавинний критерій: зміна одного біта відкритого тексту має змінювати близько половини шифротексту. Виміряємо цю величину для досліджуваного шифру, змінюючи один символ відкритого тексту та підраховуючи кількість змінених символів шифротексту (по 120 випробувань на кожну точку). Результати наведено на рисунку 2.10 і в таблиці 2.7.

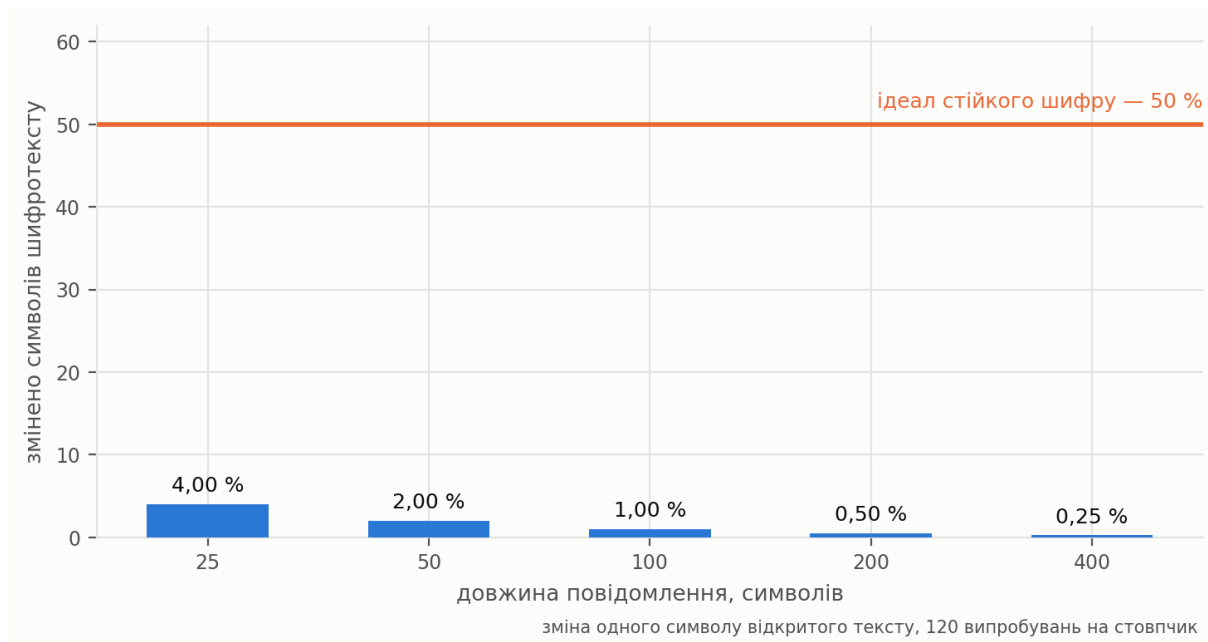


Рисунок 2.10 – Дифузія відсутня: один символ впливає рівно на один

Таблиця 2.7 – Лавинний ефект

Довжина повідомлення, символів	25	50	100	200	400
Змінено символів шифротексту	1	1	1	1	1
Частка зміненого тексту, %	4,00	2,00	1,00	0,50	0,25

Результат однозначний: змінюється **рівно один** символ, і так для повідомлень будь-якої довжини. Для тексту з 400 символів це 0,25 % замість очікуваних 50 %. Дифузія відсутня повністю — кожен символ шифрується незалежно від решти. Практичні наслідки серйозні: зловмисник може редагувати шифротекст посимвольно, а часткове знання ключа одразу дає частковий відкритий текст, без «розсіпання» решти повідомлення.

Порівняння зі справжньою машиною «Енігма»

Під час роботи виявлено цікавий факт: набір роторів з умови задачі — це справжні проводки роторів машини Enigma I (Wehrmacht, 1930 рік), але перелічені у зворотному порядку. Ротору, названому в умові «**ROTOR I**», відповідає історичний ротор III, «**ROTOR II**» — історичний ротор II, а «**ROTOR III**» — історичний ротор I.

Попри це, розглянута модель не є машиною «Енігма», а лише її суттєвим спрощенням. Основні відмінності зведено в таблиці 2.8.

Таблиця 2.8 – Порівняння справжньої Enigma I та моделі із задачі

Властивість	Справжня Enigma I	Модель із задачі
Обертання роторів	так, після кожного символу	ні, ротори нерухомі
Період шифру	$26 \cdot 25 \cdot 26 = 16\,900$	26
Рефлектор	так (UKW-B/C)	немає
Комутаційна панель	так, 10 пар кабелів	немає
Вибір і порядок роторів	3 з 5, тобто 60 варіантів	задається на вході
Початкові положення	$26^3 = 17\,576$	один зсув, 26
Самошифрування літери	неможливе (вада рефлектора)	можливе

Виникає парадокс, який варто підкреслити окремо: за «сирим» розміром простору ключів спрощена модель виглядає навіть сильнішою за історичну машину. Проте історичну «Енігму» зламували саме через її структурні вади — наявність рефлектора й неможливість переходу літери саму в себе, — а розглянуту модель ламає відсутність руху роторів, яка

перетворює її на моноалфавітну заміну з передбачуваною надбудовою.
Розмір ключа сам собою не гарантує нічого.

Висновки

1. Реалізовано шифр, що поєднує два базові криптографічні примітиви — перестановку (інкрементний зсув Цезаря) і заміну (каскад із трьох роторів). Розв’язок надіслано на платформу CodinGame й отримано оцінку валідаторів 100 %: пройдено всі шість офіційних тестів, включно з прихованими. Реалізацію продубльовано двома мовами (Python і C#) та покрито 86 автоматичними тестами, які виконуються в системі безперервної інтеграції.

2. Алгоритм оптимізовано не мікрооптимізаціями, а структурно: розуміння того, що композиція роторів є однією підстановкою, дає лінійну складність $O(L)$ без арифметичних операцій у внутрішньому циклі та сталий додатковий пам’ять.

3. Головний теоретичний результат роботи: два примітиви, застосовані послідовно й незалежно, не додають стійкості один одному. Заміна дає 26! варіантів, зсув — 26, але оскільки зсув детермінований і публічний, сумарна стійкість дорівнює стійкості однієї заміни.

4. Кількісно: заявлений простір ключів $2^{269,8}$ фактично дорівнює $2^{88,4}$, а в умовах задачі (опубліковані ротори) — лише $2^{4,7}$, тобто менш ніж 5 біт. Надлишковість у $2^{181,5}$ ключів доведено конструктивно побудованою колізією.

5. Шифр зламується в усіх чотирьох моделях атак. Найдешевша атака (підібраний відкритий текст) відновлює повний ключ за один запит із 26 символів. Найслабша за припущеннями атака — лише за шифротекстом, без будь-якого знання ключа — відновлює текст за приблизно секунду для повідомлень від 160 символів.

6. Основна вада конструкції — нерухомі ротори. Саме обертання роторів робило історичну «Енігму» поліалфавітним шифром із періодом 16 900; без нього лишається моноалфавітна заміна, замаскована передбачуваним позиційним зсувом.

7. Практичний урок роботи: висока ентропія шифротексту (4,689 з 4,700 біта на символ) і «випадковий» вигляд частотного розподілу не є доказом стійкості. Стійкість визначає структура ключа, а не зовнішній вигляд виходу шифратора.

Список використаних джерел

1. CodinGame. Encryption/Decryption of Enigma Machine [Електронний ресурс]. – Режим доступу: <https://www.codingame.com/training/easy/encryption-decryption-of-enigma-machine>
2. Caesar cipher [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Caesar_cipher
3. Enigma machine [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Enigma_machine
4. Enigma rotor details [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Enigma_rotor_details
5. Index of coincidence [Електронний ресурс] // Wikipedia. – Режим доступу: https://en.wikipedia.org/wiki/Index_of_coincidence
6. Shannon C. E. Communication Theory of Secrecy Systems // Bell System Technical Journal. – 1949. – Vol. 28, No. 4. – P. 656–715.
7. Beker H., Piper F. Cipher Systems: The Protection of Communications. – London: Northwood Books, 1982. – 436 p.
8. Singh S. The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography. – New York: Anchor Books, 2000. – 411 p.
9. Вихідний код роботи [Електронний ресурс]. – Режим доступу: <https://github.com/RE22GDV/LR2-Enigma>

Додаток А

Розв'язок задачі, надісланий на платформу CodinGame

```
"""
CodinGame - "Encryption/Decryption of Enigma Machine" (Easy)
https://www.codingame.com/training/easy/encryptiondecryption-of-enigma-machine

Self-contained solution, submitted as-is (validator score: 100%).
This file is kept ASCII-only and byte-identical to the accepted submission;
the Ukrainian write-up lives in README.md and in src/enigma/.

Input format:
  line 1    : ENCODE | DECODE
  line 2    : starting shift N
  lines 3-5 : three rotors (26 uppercase letters each)
  line 6    : the message

Encryption:
   $c[i] = S[(m[i] + N + i) \bmod 26]$ ,  $S = R3 \cdot R2 \cdot R1$ 
Decryption (exact inverse):
   $m[i] = (S^{-1}[c[i]] - N - i) \bmod 26$ 

The three rotors are composed into a single substitution S *before* the
message loop, because a composition of bijections is again a bijection.
That leaves exactly one table lookup per character:
time  $O(L + 26k)$ , extra memory  $O(26)$ .
"""

import sys

A = ord("A")
M = 26

def main() -> None:
    data = sys.stdin.read().split()
    mode = data[0].upper()
    shift = int(data[1]) % M
    rotors = data[2:5]
    message = data[5] if len(data) > 5 else ""

    # Collapse the rotor cascade into one substitution table.
    s = list(range(M))
    for wiring in rotors:
        table = [ord(ch) - A for ch in wiring]
        s = [table[x] for x in s]

    if mode == "ENCODE":
        out = [
            chr(A + s[(ord(ch) - A + shift + i) % M])
            for i, ch in enumerate(message)
        ]
    else:
        inv = [0] * M
        for i, v in enumerate(s):
            inv[v] = i
        out = [
            chr(A + (inv[ord(ch) - A] - shift - i) % M)
            for i, ch in enumerate(message)
        ]

    print("".join(out))
```

```
if __name__ == "__main__":  
    main()
```

Додаток Б

Ядро шифру (модуль src/enigma/core.py)

```
"""
Ядро шифру: два класичні криптографічні примітиви в чистому вигляді.

1. ЗСУВ (перестановка алфавіту) – шифр Цезаря з інкрементним ключем:
   i-й символ зсувається на  $(N + i) \bmod 26$ .

2. ЗАМІНА – три ротори, кожен з яких є довільною підстановкою
   (бієкцією) алфавіту A..Z.

Формально, для відкритого тексту  $m[0..L-1]$  і початкового зсуву  $N$ :


$$c[i] = S[ (m[i] + N + i) \bmod 26 ], \quad S = R3 \cdot R2 \cdot R1$$


де  $S$  – композиція трьох роторів. Зворотне перетворення:


$$m[i] = ( S^{-1}[c[i]] - N - i ) \bmod 26$$


КЛЮЧОВЕ СПОСТЕРЕЖЕННЯ (див. README, розділ «Криптоаналіз»):
композиція трьох бієкцій – це знову одна бієкція, тому три ротори
не додають стійкості порівняно з одним. Більше того, константу  $N$ 
можна «втопити» в цю бієкцію, отримавши ЄДИНИЙ ефективний ключ


$$D[x] = ( S^{-1}[x] - N ) \bmod 26, \quad m[i] = ( D[c[i]] - i ) \bmod 26$$


Саме  $D$  і відновлюють усі реалізовані атаки.
"""

from __future__ import annotations

from typing import Iterable, Sequence

#: Робочий алфавіт задачі – лише великі латинські літери.
ALPHABET = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

#: Потужність алфавіту (модуль кільця  $Z_{26}$ ).
M = 26

_A = ord("A")

class RotorError(ValueError):
    """Ротор не є коректною підстановкою алфавіту A..Z."""

class MessageError(ValueError):
    """Повідомлення містить символи поза алфавітом A..Z."""

# ----- #
# Допоміжні функції над підстановками  $Z_{26} \rightarrow Z_{26}$ 
# ----- #

def invert(perm: Sequence[int]) -> tuple[int, ...]:
    """Обернена підстановка: ``invert(p)[p[i]] == i``."""
    out = [0] * len(perm)
    for i, v in enumerate(perm):
        out[v] = i
    return tuple(out)
```

```

def perm_to_str(perm: Sequence[int]) -> str:
    """Підстановка у вигляді 26-літерного рядка (як записують ротори)."""
    return "".join(chr(_A + v) for v in perm)

def str_to_perm(text: str) -> tuple[int, ...]:
    """Рядок роторної проводки -> кортеж індексів."""
    return tuple(ord(ch) - _A for ch in text)

def normalize(text: str) -> str:
    """Прибирає все, крім латинських літер, і переводить у верхній регістр."""
    return "".join(ch for ch in text.upper() if "A" <= ch <= "Z")

def _check_message(message: str) -> None:
    bad = sorted({ch for ch in message if not ("A" <= ch <= "Z")})
    if bad:
        raise MessageError(
            "повідомлення має складатися лише з великих літер A..Z; "
            "знайдено сторонні символи: " + repr(bad)
        )

# ----- #
# Ротор
# ----- #

class Rotor:
    """
    Один ротор = одна таблиця заміни.

    Проводка задається рядком з 26 літер: літера на позиції ``i``
    визначає, у що переходить ``i``-та літера алфавіту.

    >>> Rotor("BDFHJLCPRTXVZNYEIWGAQMUSQO").encode_char("A")
    'B'
    """

    __slots__ = ("wiring", "forward", "backward")

    def __init__(self, wiring: str | Sequence[int]) -> None:
        if not isinstance(wiring, str):
            wiring = perm_to_str(wiring)
        w = wiring.strip().upper()
        if len(w) != M:
            raise RotorError(
                "ротор має містити рівно %d літер, отримано %d: %r" % (M, len(w), w)
            )
        if set(w) != set(ALPHABET):
            missing = sorted(set(ALPHABET) - set(w))
            dup = sorted({c for c in w if w.count(c) > 1})
            raise RotorError(
                "ротор %r не є підстановкою алфавіту "
                "(відсутні: %s, дублікати: %s)" % (w, missing, dup)
            )
        self.wiring = w
        self.forward = str_to_perm(w)
        self.backward = invert(self.forward)

# -- пряме / зворотнє відображення ----- #

    def encode_char(self, ch: str) -> str:
        return chr(_A + self.forward[ord(ch) - _A])

    def decode_char(self, ch: str) -> str:

```

```

        return chr(_A + self.backward[ord(ch) - _A])

def encode(self, text: str) -> str:
    f = self.forward
    return "".join(chr(_A + f[ord(c) - _A]) for c in text)

def decode(self, text: str) -> str:
    b = self.backward
    return "".join(chr(_A + b[ord(c) - _A]) for c in text)

# -- сервіс ----- #

def inverse(self) -> "Rotor":
    """Ротор, що реалізує обернену підстановку."""
    return Rotor(perm_to_str(self.backward))

@property
def fixed_points(self) -> tuple[int, ...]:
    """Літери, які ротор залишає на місці (`S[x] == x`)."""
    return tuple(i for i, v in enumerate(self.forward) if i == v)

@property
def cycle_type(self) -> tuple[int, ...]:
    """Циклова структура підстановки – довжини циклів за спаданням."""
    seen = [False] * M
    cycles: list[int] = []
    for start in range(M):
        if seen[start]:
            continue
        length, cur = 0, start
        while not seen[cur]:
            seen[cur] = True
            cur = self.forward[cur]
            length += 1
        cycles.append(length)
    return tuple(sorted(cycles, reverse=True))

def __eq__(self, other: object) -> bool:
    return isinstance(other, Rotor) and other.wiring == self.wiring

def __hash__(self) -> int:
    return hash(self.wiring)

def __repr__(self) -> str:
    return "Rotor(%r)" % (self.wiring,)

def __str__(self) -> str:
    return self.wiring

# ----- #
# Машина
# ----- #

class EnigmaMachine:
    """
    Спрощена Енігма: інкрементний зсув Цезаря + каскад роторів.

    :param rotors: послідовність роторів (рядки або ``Rotor``); порядок –
        у якому сигнал проходить крізь них.
    :param shift: початкове значення зсуву ``N`` (0 <= N < 26).

    >>> m = EnigmaMachine(["BDFHJLCPRTXVZNYEIWGAQMUSQO",
    ...                     "AJDKSIRUXBLHWTMCQGZNPYFVOE",
    ...                     "EKMFLGDQVZNTOWYHXUSPAIBRCJ"], shift=4)
    >>> m.encode("AAA")

```

```

'KQF'
>>> m.decode("KQF")
'AAA'
"""

__slots__ = ("rotors", "shift", "substitution", "inverse_substitution",
             "_enc_tables", "_dec_tables")

def __init__(self, rotors: Iterable[str | Rotor], shift: int = 0) -> None:
    self.rotors: tuple[Rotor, ...] = tuple(
        r if isinstance(r, Rotor) else Rotor(r) for r in rotors
    )
    if not self.rotors:
        raise RotorError("потрібен щонайменше один ротор")
    self.shift = int(shift) % M

    # --- згортка каскаду роторів в одну підстановку S -----
    # Саме тут видно, що «три таблиці заміни» = «одна таблиця заміни».
    composed = list(range(M))
    for rotor in self.rotors:
        composed = [rotor.forward[x] for x in composed]
    self.substitution: tuple[int, ...] = tuple(composed)
    self.inverse_substitution: tuple[int, ...] = invert(self.substitution)

    # --- 26 готових алфавітів шифрування -----
    # Шифр є періодичним поліалфавітним з періодом 26: на позиції i
    # застосовується алфавіт номер (N + i) mod 26. Завдяки попередньому
    # обчисленню таблиць внутрішній цикл не містить жодної арифметики.
    s, inv = self.substitution, self.inverse_substitution
    self._enc_tables: tuple[bytes, ...] = tuple(
        bytes(_A + s[(m + p) % M] for m in range(M)) for p in range(M)
    )
    self._dec_tables: tuple[bytes, ...] = tuple(
        bytes(_A + (inv[c] - p) % M for c in range(M)) for p in range(M)
    )

# -- основне API ----- #

def encode(self, message: str, *, validate: bool = True) -> str:
    """Зашифрувати повідомлення (лише літери A..Z)."""
    if validate:
        _check_message(message)
    tables, n = self._enc_tables, self.shift
    return bytes(
        tables[(n + i) % M][ord(ch) - _A] for i, ch in enumerate(message)
    ).decode("ascii")

def decode(self, ciphertext: str, *, validate: bool = True) -> str:
    """Розшифрувати повідомлення (лише літери A..Z)."""
    if validate:
        _check_message(ciphertext)
    tables, n = self._dec_tables, self.shift
    return bytes(
        tables[(n + i) % M][ord(ch) - _A] for i, ch in enumerate(ciphertext)
    ).decode("ascii")

def process(self, mode: str, message: str) -> str:
    """Диспетчер за режимом ``ENCODE`` / ``DECODE`` (формат CodinGame)."""
    mode_u = mode.strip().upper()
    if mode_u == "ENCODE":
        return self.encode(message)
    if mode_u == "DECODE":
        return self.decode(message)
    raise ValueError(
        "невідомий режим %r: очікується ENCODE або DECODE" % (mode,)
    )

```

```

# -- структурні властивості (використовуються в криптоаналізі) ----- #

@property
def effective_key(self) -> tuple[int, ...]:
    """
    Ефективний ключ ``D``: єдина бієкція, що повністю задає шифр.

    
$$m[i] = (D[c[i]] - i) \bmod 26$$


    Увесь номінальний ключ (3 ротори + зсув) стискається до цих 26 чисел.
    """
    n = self.shift
    return tuple((v - n) % M for v in self.inverse_substitution)

@property
def alphabets(self) -> tuple[str, ...]:
    """26 алфавітів заміни, які шифр циклічно застосовує до позицій."""
    return tuple(t.decode("ascii") for t in self._enc_tables)

def equivalent_single_rotor(self) -> "EnigmaMachine":
    """
    Машина з ОДНИМ ротором і нульовим зсувом, еквівалентна цій.

    Доводить, що каскад «3 ротори + зсув N» не має жодної переваги
    над «1 ротор + зсув 0».
    """
    return EnigmaMachine([Rotor(perm_to_str(invert(self.effective_key)))], shift=0)

def __repr__(self) -> str:
    rotors = ", ".join(r.wiring for r in self.rotors)
    return "EnigmaMachine(shift=%d, rotors=%s)" % (self.shift, rotors)

# ----- #
# Робота напряму з ефективним ключем D (потрібна атакам)
# ----- #

def decrypt_with_effective_key(ciphertext: str, key: Sequence[int]) -> str:
    """``m[i] = (D[c[i]] - i) mod 26``"""
    return bytes(
        _A + (key[ord(ch) - _A] - i) % M for i, ch in enumerate(ciphertext)
    ).decode("ascii")

def encrypt_with_effective_key(plaintext: str, key: Sequence[int]) -> str:
    """Обернена до :func:`decrypt_with_effective_key` операція."""
    inv = invert(key)
    return bytes(
        _A + inv[(ord(ch) - _A + i) % M] for i, ch in enumerate(plaintext)
    ).decode("ascii")

```

Додаток В

Криптоаналітичні атаки (модуль src/enigma/attacks.py)

```
"""
Криптоаналіз спрощеної Енігми.

Реалізовано чотири атаки, впорядковані за силою припущень про
можливості зломисника (класична ієрархія моделей атак):

1. :func:`brute_force_shift` - відомі ротори, невідомий зсув.
    Простір ключів = 26. Секунди -> мілісекунди.
2. :func:`known_plaintext_attack` - відома пара (текст, шифротекст).
    Лінійне відновлення ключа, без перебору.
3. :func:`chosen_plaintext_attack` - доступ до шифрувального оракула.
    ОДИН запит із 26 символів -> повний ключ.
4. :func:`ciphertext_only_attack` - відомий лише шифротекст. Сходження
    за схилом по квадриграмах + жадібна
    частотна ініціалізація.

Усі атаки відновлюють той самий об'єкт - ефективний ключ ``D`` (26 чисел),
див. :attr:`enigma.core.EnigmaMachine.effective_key`.

Модуль не має зовнішніх залежностей: лише стандартна бібліотека.
"""

from __future__ import annotations

import math
import random
import time
from dataclasses import dataclass, field
from typing import Callable, Sequence

from .analysis import ENGLISH_FREQ
from .core import (
    ALPHABET,
    M,
    EnigmaMachine,
    Rotor,
    decrypt_with_effective_key,
    invert,
)
from .ngrams import NgramScorer, default_scorer

_A = ord("A")

# ----- #
# Результати
# ----- #

@dataclass
class AttackResult:
    """Уніфікований результат будь-якої атаки."""

    name: str
    success: bool
    plaintext: str
    key: tuple[int, ...] | None
    score: float
    elapsed_s: float
    evaluations: int = 0
    notes: dict[str, object] = field(default_factory=dict)
```

```

def key_letters(self) -> str | None:
    """Ефективний ключ у літерному вигляді (для друку)."""
    if self.key is None:
        return None
    return "".join(chr(_A + v) for v in self.key)

def __str__(self) -> str:
    status = "УСПІХ" if self.success else "невдача"
    return "%-28s %-8s score=%9.1f t=%6.3f с %s" % (
        self.name, status, self.score, self.elapsed_s, self.plaintext[:48]
    )

# ----- #
# Оцінка простору ключів
# ----- #

def keyspace_analysis(rotor_count: int = 3) -> dict[str, float]:
    """
    Номінальний і фактичний розмір простору ключів, у бітах.

    Номінально ключ — це ``rotor_count`` довільних підстановок плюс зсув:
    ``(26!)^k * 26``. Фактично ж усе згортається в одну підстановку ``D``,
    тому справжня стійкість дорівнює ``26!`` незалежно від k.
    """
    bits_perm = math.log2(math.factorial(M))
    nominal = rotor_count * bits_perm + math.log2(M)
    effective = bits_perm
    return {
        "bits_per_rotor": bits_perm,
        "nominal_bits": nominal,
        "effective_bits": effective,
        "reduction_bits": nominal - effective,
        "equivalent_keys_bits": nominal - effective,
        # Сценарій за принципом Керкгоффа: ротори опубліковані в умові
        # задачі й однакові в усіх тестах, тож секретом є тільки N.
        "kerckhoffs_bits": math.log2(M),
    }

def find_key_collision(
    rng: random.Random | int | None = None, tries: int = 200
) -> dict[str, object] | None:
    """
    Конструктивний доказ надлишковості ключа: знаходить ДВА різні набори
    (ротори + зсув), які шифрують будь-яке повідомлення однаково.

    Побудова не випадкова, а пряма: беремо довільні R1, R2, R3, N і
    замінюємо трійку на еквівалентний один ротор зі зсувом 0.
    """
    from .rotors import random_rotor_set

    if not isinstance(rng, random.Random):
        rng = random.Random(rng)

    for _ in range(tries):
        rotors = random_rotor_set(3, rng)
        shift = rng.randrange(M)
        a = EnigmaMachine(rotors, shift)
        b = a.equivalent_single_rotor()
        probe = "".join(rng.choice(ALPHABET) for _ in range(120))
        if a.encode(probe) == b.encode(probe) and a.rotors != b.rotors:
            return {
                "key_a": {"rotors": [r.wiring for r in a.rotors], "shift": a.shift},
                "key_b": {"rotors": [r.wiring for r in b.rotors], "shift": b.shift},
            }

```

```

        "probe": probe,
        "ciphertext": a.encode(probe),
        "identical": True,
    }
    return None

# ----- #
# Атака 1: перебір зсуву (ротори відомі)
# ----- #

def brute_force_shift(
    ciphertext: str,
    rotors: Sequence[str | Rotor],
    scorer: NgramScorer | None = None,
) -> AttackResult:
    """
    Повний перебір усіх 26 значень зсуву за відомих роторів.

    Це реалістичний сценарій саме для цієї задачі: ротори надруковані
    в умові й однакові в усіх шести офіційних тестах, тому секретом
    лишається тільки N – трохи менше ніж 5 біт ентропії.
    """
    scorer = scorer or default_scorer()
    t0 = time.perf_counter()
    candidates: list[tuple[float, int, str]] = []
    for n in range(M):
        pt = EnigmaMachine(rotors, n).decode(ciphertext)
        candidates.append((scorer.score(pt), n, pt))
    candidates.sort(reverse=True)
    best_score, best_n, best_pt = candidates[0]
    elapsed = time.perf_counter() - t0

    margin = best_score - candidates[1][0] if len(candidates) > 1 else 0.0
    key = EnigmaMachine(rotors, best_n).effective_key
    return AttackResult(
        name="Перебір зсуву (26 ключів)",
        success=True,
        plaintext=best_pt,
        key=key,
        score=best_score,
        elapsed_s=elapsed,
        evaluations=M,
        notes={
            "recovered_shift": best_n,
            "margin_over_runner_up": margin,
            "ranking": [(n, round(s, 2), p) for s, n, p in candidates[:5]],
        },
    )

# ----- #
# Атака 2: відомий відкритий текст
# ----- #

def known_plaintext_attack(
    plaintext: str, ciphertext: str, *, full_ciphertext: str | None = None
) -> AttackResult:
    """
    Атака на основі відомого фрагмента («шпаргалки», crib).

    З кожної пари ``(m[i], c[i])`` одразу впливає один елемент ключа:

        
$$D[c[i]] = (m[i] + i) \bmod 26$$


    Жодного перебору – чисте лінійне відновлення. Щоб покрити всі 26

```

```

елементів, потрібно в середньому  $26 \cdot H(26) \sim 100$  символів (задача
про збирання купонів); часткового ключа вже достатньо, щоб
прочитати більшу частину повідомлення.
"""
t0 = time.perf_counter()
n_pairs = min(len(plaintext), len(ciphertext))
partial: list[int | None] = [None] * M
contradictions = 0

for i in range(n_pairs):
    c = ord(ciphertext[i]) - _A
    value = (ord(plaintext[i]) - _A + i) % M
    if partial[c] is None:
        partial[c] = value
    elif partial[c] != value:
        contradictions += 1

known = [v for v in partial if v is not None]
coverage = len(known)

# Доповнюємо невідомі позиції довільною бієкцією, щоб ключ був повним.
unused = [v for v in range(M) if v not in set(known)]
completed = []
it = iter(unused)
for v in partial:
    completed.append(v if v is not None else next(it))
key = tuple(completed)

target = full_ciphertext if full_ciphertext is not None else ciphertext
recovered = decrypt_with_effective_key(target, key)
elapsed = time.perf_counter() - t0

return AttackResult(
    name="Відомий відкритий текст",
    success=contradictions == 0 and coverage == M,
    plaintext=recovered,
    key=key,
    score=float(coverage),
    elapsed_s=elapsed,
    evaluations=n_pairs,
    notes={
        "crib_length": n_pairs,
        "key_coverage": coverage,
        "coverage_percent": 100.0 * coverage / M,
        "contradictions": contradictions,
        "unknown_letters": [ALPHABET[i] for i, v in enumerate(partial) if v is None],
    },
)

def expected_crib_coverage(crib_length: int) -> float:
    """
    Теоретичне очікуване покриття ключа за довжиною шпаргалки
    (модель рівномірних незалежних літер шифротексту):

    
$$E[\text{coverage}] = 26 * (1 - (25/26)^L)$$

    """
    return M * (1.0 - ((M - 1) / M) ** crib_length)

# ----- #
# Атака 3: підібраний відкритий текст
# ----- #

def chosen_plaintext_attack(
    oracle: Callable[[str], str], ciphertext: str | None = None

```

```

) -> AttackResult:
    """
    Найсильніша й найдешевша атака: маючи доступ до шифрувального
    оракула, повний ключ дістається за ОДИН запит із 26 літер.

    Надсилаємо ``"AAAA...A"`` (26 символів). Тоді
    ``c[i] = S[(0 + N + i) mod 26]``, а отже ``D[c[i]] = i``.
    Оскільки S — бієкція, усі 26 символів ``c[i]`` різні, і ключ
    відновлюється повністю.
    """
    t0 = time.perf_counter()
    probe = "A" * M
    response = oracle(probe)
    if len(response) < M or len(set(response)) != M:
        raise ValueError("оракул повернув некоректну відповідь на пробу")

    key_list = [0] * M
    for i, ch in enumerate(response[:M]):
        key_list[ord(ch) - _A] = i
    key = tuple(key_list)

    recovered = decrypt_with_effective_key(ciphertext, key) if ciphertext else ""
    elapsed = time.perf_counter() - t0
    return AttackResult(
        name="Підібраний відкритий текст",
        success=True,
        plaintext=recovered,
        key=key,
        score=float(M),
        elapsed_s=elapsed,
        evaluations=1,
        notes={"queries": 1, "query_length": M, "probe": probe, "response": response},
    )

# ----- #
# Атака 4: тільки шифротекст
# ----- #

def frequency_initial_key(ciphertext: str) -> list[int]:
    """
    Жадібна частотна ініціалізація ключа.

    Для кожної літери шифротексту ``x`` і кожного кандидата ``v``
    рахуємо, наскільки «англійськими» за частотою літер вийдуть
    символи ``(v - i) mod 26`` на всіх позиціях, де стоїть ``x``.
    Потім жадібно розбираємо пари (x, v) за спаданням вигоди,
    зберігаючи бієктивність.
    """
    log_freq = [math.log(max(ENGLISH_FREQ[ch], 1e-3) / 100.0) for ch in ALPHABET]

    positions: list[list[int]] = [[] for _ in range(M)]
    for i, ch in enumerate(ciphertext):
        positions[ord(ch) - _A].append(i)

    gains: list[tuple[float, int, int]] = []
    for x in range(M):
        pos = positions[x]
        if not pos:
            continue
        for v in range(M):
            gains.append((sum(log_freq[(v - i) % M] for i in pos), x, v))
    gains.sort(reverse=True)

    key: list[int | None] = [None] * M
    used_v: set[int] = set()

```

```

for _, x, v in gains:
    if key[x] is None and v not in used_v:
        key[x] = v
        used_v.add(v)

leftovers = [v for v in range(M) if v not in used_v]
it = iter(leftovers)
return [v if v is not None else next(it) for v in key]

def ciphertext_only_attack(
    ciphertext: str,
    *,
    restarts: int = 24,
    patience: int = 1500,
    scorer: NgramScorer | None = None,
    rng: random.Random | int | None = None,
    init: str = "frequency",
    expected_plaintext: str | None = None,
) -> AttackResult:
    """
    Атака лише на шифротексті: сходження за схилом у просторі бієкцій ``D``
    з фітнес-функцією на квадриграмах англійської мови.

    Ключова ідея. Позиційний зсув ``-i`` є ПУБЛІЧНОЮ структурою, а не
    секретом, тому його не треба вгадувати: перебирається лише бієкція
    ``D`` (26! варіантів). Через це стійкість шифру дорівнює стійкості
    звичайної моноалфавітної заміни, попри «поліалфавітний» вигляд.

    :param restarts: кількість незалежних запусків із різних початкових точок
    :param patience: скільки невдалих випадкових обмінів поспіль терпіти
    :param init: ``"frequency"`` (жадібна частотна) або ``"random"``
    """
    scorer = scorer or default_scorer()
    if not isinstance(rng, random.Random):
        rng = random.Random(rng)

    t0 = time.perf_counter()
    cipher_idx = [ord(ch) - _A for ch in ciphertext]
    length = len(cipher_idx)
    evaluations = 0

    def decrypt_idx(key: Sequence[int]) -> list[int]:
        return [(key[c] - i) % M for i, c in enumerate(cipher_idx)]

    best_key: list[int] = list(range(M))
    best_score = -float("inf")

    for attempt in range(restarts):
        if init == "frequency" and attempt == 0:
            key = frequency_initial_key(ciphertext)
        elif init == "frequency" and attempt < max(2, restarts // 4):
            # Кілька збурених копій частотного старту.
            key = frequency_initial_key(ciphertext)
            for _ in range(4):
                a, b = rng.randrange(M), rng.randrange(M)
                key[a], key[b] = key[b], key[a]
        else:
            key = list(range(M))
            rng.shuffle(key)

        score = scorer.score_indices(decrypt_idx(key))
        evaluations += 1
        stale = 0
        while stale < patience:
            a = rng.randrange(M)

```

```

        b = rng.randrange(M)
        if a == b:
            continue
        key[a], key[b] = key[b], key[a]
        trial = scorer.score_indices(decrypt_idx(key))
        evaluations += 1
        if trial > score:
            score = trial
            stale = 0
        else:
            key[a], key[b] = key[b], key[a] # відкат
            stale += 1

    if score > best_score:
        best_score, best_key = score, list(key)

plaintext = decrypt_with_effective_key(ciphertext, best_key)
elapsed = time.perf_counter() - t0

notes: dict[str, object] = {
    "restarts": restarts,
    "patience": patience,
    "init": init,
    "score_per_char": best_score / length if length else 0.0,
}
success = True
if expected_plaintext is not None:
    matches = sum(1 for a, b in zip(plaintext, expected_plaintext) if a == b)
    accuracy = matches / max(len(expected_plaintext), 1)
    notes["accuracy"] = accuracy
    notes["exact"] = plaintext == expected_plaintext
    success = accuracy >= 0.95

return AttackResult(
    name="Тільки шифротекст (hill climbing)",
    success=success,
    plaintext=plaintext,
    key=tuple(best_key),
    score=best_score,
    elapsed_s=elapsed,
    evaluations=evaluations,
    notes=notes,
)

# ----- #
# Діагностика дифузії
# ----- #

def avalanche_test(
    machine: EnigmaMachine,
    length: int = 200,
    trials: int = 300,
    rng: random.Random | int | None = None,
) -> dict[str, float]:
    """
    Лавинний ефект: яка частка символів шифротексту змінюється,
    якщо змінити ОДИН символ відкритого тексту.

    Для стійкого блокового шифру очікується ~50 %. Тут очікується
    рівно один змінений символ, тобто ``1/L`` – дифузія відсутня
    повністю, кожен символ шифрується незалежно від решти.
    """
    if not isinstance(rng, random.Random):
        rng = random.Random(rng)

```

```

changed_total = 0
for _ in range(trials):
    pt = [rng.choice(ALPHABET) for _ in range(length)]
    base = machine.encode("".join(pt))
    pos = rng.randrange(length)
    original = pt[pos]
    pt[pos] = rng.choice([c for c in ALPHABET if c != original])
    mutated = machine.encode("".join(pt))
    changed_total += sum(1 for a, b in zip(base, mutated) if a != b)

avg_changed = changed_total / trials
return {
    "message_length": float(length),
    "trials": float(trials),
    "avg_changed_chars": avg_changed,
    "avalanche_ratio": avg_changed / length,
    "ideal_ratio": 0.5,
}

def run_all_attacks(
    machine: EnigmaMachine,
    plaintext: str,
    *,
    crib_length: int = 40,
    restarts: int = 24,
    rng: random.Random | int | None = None,
) -> list[AttackResult]:
    """Прогін усіх чотирьох атак на одному повідомленні – зручно для звіту."""
    ciphertext = machine.encode(plaintext)
    results = [
        brute_force_shift(ciphertext, machine.rotors),
        known_plaintext_attack(
            plaintext[:crib_length], ciphertext[:crib_length],
            full_ciphertext=ciphertext,
        ),
        chosen_plaintext_attack(machine.encode, ciphertext),
        ciphertext_only_attack(
            ciphertext, restarts=restarts, rng=rng, expected_plaintext=plaintext
        ),
    ]
    truth = machine.effective_key
    for r in results:
        r.notes["key_exact_match"] = r.key == truth
        if r.name.startswith("Відомий"):
            r.success = r.plaintext == plaintext
    return results

```

Додаток Г

Незалежна реалізація мовою C# (csharp/Enigma/Program.cs)

```
// Лабораторна робота №2 – «Захист даних»
// CodinGame: Encryption/Decryption of Enigma Machine
//
// Друга, незалежна реалізація того самого алгоритму мовою C#.
// Призначення – перехресна перевірка: обидві реалізації мають давати
// побітово однаковий результат на всіх шести офіційних тестах.
//
// Запуск:
// dotnet run --project csharp/Enigma -- selftest
// dotnet run --project csharp/Enigma < input.txt (режим CodinGame)
// dotnet run --project csharp/Enigma -- encode 4 AAA

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Lab2.Enigma;

/// <summary>Один ротор – таблиця заміни (підстановка алфавіту A..Z).</summary>
public sealed class Rotor
{
    public const int M = 26;
    private const int A = 'A';

    public string Wiring { get; }
    public int[] Forward { get; }
    public int[] Backward { get; }

    public Rotor(string wiring)
    {
        wiring = (wiring ?? string.Empty).Trim().ToUpperInvariant();
        if (wiring.Length != M)
            throw new ArgumentException($"Ротор має містити рівно {M} літер, отримано {wiring.Length}");
        if (wiring.Distinct().Count() != M || wiring.Any(c => c < 'A' || c > 'Z'))
            throw new ArgumentException($"Ротор '{wiring}' не є підстановкою алфавіту A..Z.");

        Wiring = wiring;
        Forward = wiring.Select(c => c - A).ToArray();
        Backward = new int[M];
        for (int i = 0; i < M; i++) Backward[Forward[i]] = i;
    }
}

/// <summary>
/// Спрощена машина Енігми: інкрементний зсув Цезаря + каскад роторів.
/// c[i] = S[(m[i] + N + i) mod 26], S = R3 . R2 . R1
/// </summary>
public sealed class EnigmaMachine
{
    private const int M = 26;
    private const int A = 'A';

    private readonly int[] _s; // композиція роторів
    private readonly int[] _sInv; // обернена підстановка
    private readonly int _shift;

    public EnigmaMachine(IEnumerable<string> rotors, int shift)
    {
        _shift = ((shift % M) + M) % M;
    }
}
```

```

        // Зпортка каскаду в одну підстановку: три таблиці заміни = одна.
        _s = Enumerable.Range(0, M).ToArray();
        foreach (var wiring in rotors)
        {
            var rotor = new Rotor(wiring);
            _s = _s.Select(x => rotor.Forward[x]).ToArray();
        }

        _sInv = new int[M];
        for (int i = 0; i < M; i++) _sInv[_s[i]] = i;
    }

    public string Encode(string message)
    {
        Validate(message);
        var sb = new StringBuilder(message.Length);
        for (int i = 0; i < message.Length; i++)
            sb.Append((char) (A + _s[(message[i] - A + _shift + i) % M]));
        return sb.ToString();
    }

    public string Decode(string ciphertext)
    {
        Validate(ciphertext);
        var sb = new StringBuilder(ciphertext.Length);
        for (int i = 0; i < ciphertext.Length; i++)
        {
            int v = (_sInv[ciphertext[i] - A] - _shift - i) % M;
            if (v < 0) v += M;
            sb.Append((char) (A + v));
        }
        return sb.ToString();
    }

    public string Process(string mode, string message) => mode.Trim().ToUpperInvariant() switch
    {
        "ENCODE" => Encode(message),
        "DECODE" => Decode(message),
        _ => throw new ArgumentException($"Невідомий режим '{mode}': очікується ENCODE або DECODE"),
    };

    private static void Validate(string text)
    {
        foreach (var ch in text)
            if (ch < 'A' || ch > 'Z')
                throw new ArgumentException($"Неприпустимий символ '{ch}': дозволені лише A..Z");
    }

    public static class Program
    {
        private static readonly string[] Rotors =
        {
            "BDFHJLCPRTXVZNYEIWGAKMUSQO",
            "AJDKSIRUXBLHWTMCQGZNPYFVOE",
            "EKMFLGDQVZNTOWYHXUSPAIBRCJ"
        };

        /// <summary>Шість офіційних валідаційних тестів CodinGame.</summary>
        private static readonly (string Label, string Mode, int Shift, string In, string Out)[] Cases =
        {
            ("Encode 3", "ENCODE", 4, "AAA", "KQF"),
            ("Encode 23", "ENCODE", 7, "WEATHERREPORTWINDYTODAY", "ALWAURKQEQQLRAWZHUYKVN"),
            ("Decode 21", "DECODE", 9, "PQSACVVTOISXFXCIAMQEM", "EVERYONEISWELCOMEHRE"),
            ("Encode 21", "ENCODE", 9, "EVERYONEISWELCOMEHRE", "PQSACVVTOISXFXCIAMQEM"),
        };
    }

```

```

        ("Encode 42", "ENCODE", 9, "EVERYONEISWELCOMHEREEVERYONEISWELCOMHERE",
        "PQSACVVTOISXFXCIAMQEMDZIXFJJSTQIENEFQXVZYV"),
        ("Decode 49", "DECODE", 5, "XPCXAUPHYQALKJMGKRWPGYHFTKRFFFNOUTZCABUAHQLGXREZ",
        "THEQUICKBROWNFOXJUMPSOVERALAZYSPIXOFBLACKQUARTZ")
    };

public static int Main(string[] args)
{
    Console.OutputEncoding = Encoding.UTF8;

    if (args.Length > 0 && args[0].Equals("selftest", StringComparison.OrdinalIgnoreCase))
        return SelfTest();

    if (args.Length >= 3)
    {
        var machine = new EnigmaMachine(Rotors, int.Parse(args[1]));
        Console.WriteLine(machine.Process(args[0], args[2].ToUpperInvariant()));
        return 0;
    }

    // Режим CodinGame: шість рядків зі стандартного входу.
    var mode = Console.ReadLine()!.Trim();
    var shift = int.Parse(Console.ReadLine()!.Trim());
    var rotors = new[] { Console.ReadLine()!.Trim(), Console.ReadLine()!.Trim(), Console.Re
    var message = (Console.ReadLine() ?? string.Empty).Trim();
    Console.WriteLine(new EnigmaMachine(rotors, shift).Process(mode, message));
    return 0;
}

private static int SelfTest()
{
    Console.WriteLine("Офіційні тести CodinGame (реалізація на C#)");
    Console.WriteLine(new string('-', 72));

    int passed = 0;
    foreach (var (label, mode, shift, input, expected) in Cases)
    {
        var got = new EnigmaMachine(Rotors, shift).Process(mode, input);
        var ok = got == expected;
        if (ok) passed++;
        Console.WriteLine($"[{(ok ? "OK" : "FAIL")}] {label,-11} {mode} -> {got}");
    }

    // Властивість: розшифрування – точна інверсія шифрування.
    var rng = new Random(2024);
    int roundTrips = 0;
    for (int t = 0; t < 2000; t++)
    {
        var len = rng.Next(1, 60);
        var msg = new string(Enumerable.Range(0, len)
            .Select(_ => (char)('A' + rng.Next(26))).ToArray());
        var m = new EnigmaMachine(Rotors, rng.Next(26));
        if (m.Decode(m.Encode(msg)) == msg) roundTrips++;
    }

    Console.WriteLine(new string('-', 72));
    Console.WriteLine($"Офіційні тести: пройдено {passed} з {Cases.Length}");
    Console.WriteLine($"Перевірка оборотності: {roundTrips} з 2000 випадкових повідомлень");
    return passed == Cases.Length && roundTrips == 2000 ? 0 : 1;
}
}

```